

# Lab 6 Assignment (CSI3120)

## Programming in Prolog

### Recursive Structures, Backtracking with Cut, and Advanced List Transformations

---

In this lab, students will build on the foundational concepts introduced in Lab 5 by exploring more advanced Prolog topics such as recursion with accumulators, controlling backtracking, and optimizing tail recursion. Through hands-on tasks, students will develop a deeper understanding of how to manage lists, control Prolog's backtracking mechanism, and work with more complex recursion patterns.

Topics Covered:

- Recursion with Accumulators: Learn how to implement recursive solutions using accumulators to store intermediate results.
- Controlling Backtracking with Cut Operator (`!/1`): Understand how to prevent unnecessary backtracking and optimize the flow of logic using the cut operator.
- List Operations: Perform list manipulation tasks, including finding, filtering, and sorting elements in a list.

The lab is divided into 3 tasks, focusing on recursive techniques, list processing, and backtracking control. All tasks must be completed in groups of three

#### **TASK A: Recursion, Dynamic Filtering, and Advanced List Manipulation (40 Points)**

**Question 1:** Write a tail-recursive Prolog predicate to compute the factorial of a non-negative integer using an accumulator. The predicate should validate the input to ensure it is a non-negative integer and return an error message for invalid inputs (e.g., negative numbers or non-integer values).

Handle invalid input: Ensure that the input is a valid non-negative integer, returning an appropriate error message for invalid inputs (e.g., negative numbers or non-integer values).

Add multiple base cases: Use a base case for `n = 0` and additional validation for `n < 0` (invalid case).

Note: Marks will be reduced if accumulator is not used.

Test Cases:

```
?- factorial(0, 1, Result).
   % Result = 1.
?- factorial(5, 1, Result).
   % Result = 120.
?- factorial(10, 1, Result).
   % Result = 3628800.
?- factorial(-1, 1, Result).
   % Result = 'Error: Input must be a non-negative integer.'
```

**Question 2:** Write a Prolog predicate `filter_list/3` that filters elements from a list based on dynamically passed conditions. The conditions will be passed as a list of predicates, and the result should include only the elements that satisfy all the conditions.

Define a Predicate **filter\_list/3**: Filters a list based on the two conditions

You are required to implement two conditions:

1. **greater\_than/2**: Filters elements greater than a given value.
2. **multiple\_of/2**: Filters elements that are multiples of a given value.

Test Cases:

```
?- filter_list([1, 2, 3, 4, 5, 6, 7, 8], [greater_than(3), multiple_of(2)], Result).
   % Result = [4, 6, 8].
?- filter_list([1, 2, 3, 6, 9, 12], [greater_than(5), multiple_of(3)], Result).
   % Result = [6, 9, 12].
?- filter_list([1, 2, 3, 4, 5], [greater_than(10), multiple_of(2)], Result).
   % Result = [].
?- filter_list([], [greater_than(3), multiple_of(2)], Result).
   % Result = [].
?- filter_list([1, 2, 3, 4, 5], [greater_than(0), multiple_of(1)], Result).
   % Result = [1, 2, 3, 4, 5].
```

**Question 3:** Write a Prolog predicate `second_max/2` that finds the second maximum element in a list. The predicate should handle the following cases:

Lists with duplicate values (e.g., if the maximum value appears multiple times, it should still find the second distinct maximum).

Lists with fewer than two distinct elements should return an appropriate error message.

Test Cases:

```
?- second_max([4, 1, 3, 7, 5], SecondMax).
   % SecondMax = 5.
?- second_max([5, 5, 3, 9, 9], SecondMax).
   % SecondMax = 5.
?- second_max([7, 7, 7, 7], SecondMax).
   % SecondMax = 'Error: List must contain at least two distinct elements.'
```

```
?- second_max([7], SecondMax).
% SecondMax = 'Error: List must contain at least two distinct elements.'.
?- second_max([], SecondMax).
% SecondMax = 'Error: List must contain at least two distinct elements.'.
?- second_max([5, -1, 3, 2, -10], SecondMax).
% SecondMax = 3.
?- second_max([4, -1, -3, 7, -5], SecondMax).
% SecondMax = 4.
```

## Report Requirements

In addition to submitting your code, please prepare a short report that addresses the following aspects of your solution for Task A, which consists of three questions:

Explanation of Recursive Solutions and Conditions:

1. Factorial with Tail Recursion and Accumulators:

- **Explanation of the Accumulator Logic:** Describe how you used an accumulator to store the intermediate results of the factorial calculation and how this helps make the recursion tail-recursive.
- **Input Validation:** Explain how your solution handles negative inputs and why this validation is necessary.

2. Filtering Elements Based on Multiple Conditions:

- **Dynamic Conditions:** Explain how you implemented the dynamic conditions `greater_than/2` and `multiple_of/2` for filtering the elements of a list.
- **Recursive Logic:** Describe how the recursive predicate `filter_list/3` processes each element of the list

3. Finding the Second Maximum Element:

- **Handling Duplicates:** Explain how your solution ensures that duplicate values do not affect finding the second maximum element.
- **Error Handling for Insufficient Elements:** Describe how your solution handles lists with fewer than two distinct elements and why this is important.

## Task B: Backtracking with Cut Operator (20 Points)

You are given a list of people, each represented by their name, height, and age (e.g., `person(Name, Height, Age)`). Your task is to write a Prolog predicate `taller_than/4` that finds the first person in the list who is taller than a specified height and age between a minimum (Inclusive) and maximum (Exclusive) range. Once a match is found, the search should stop, and no further solutions should be explored.

Use the cut operator (`!`) to ensure that backtracking halts after finding the first person who satisfies both conditions.

Ensure that your solution can handle edge cases, such as an empty list or when no one meets both conditions.

Test Cases:

```
?- taller_than([person(john, 180, 30), person(linda, 170, 29), person(sam, 165, 32)], 160, age_in_range(28, 33), Result).
```

```
% Expected Result: Result = person(john, 180, 30).
```

```
?- taller_than([person(john, 180, 30), person(linda, 170, 25), person(sam, 160, 35), person(jake, 180, 28)], 170, age_in_range(25, 30), Result).
```

```
% Expected Result: Result = person(jake, 180, 28).
```

```
?- taller_than([person(john, 180, 30), person(linda, 170, 25), person(sam, 160, 35)], 190, age_in_range(25, 35), Result).
```

```
% Result = 'No match found.'.
```

```
?- taller_than([], 170, age_in_range(25, 35), Result).
```

```
% Result = 'No match found.'.
```

## Report Requirements

In addition to submitting your code, please prepare a brief report that addresses the following points:

1. Explain how you implemented backtracking control using the cut (!) operator to stop the search after finding the first valid match.
2. Describe how your code processes the input list of `person(Name, Height, Age)` to apply both the height and fixed age conditions.
3. Explain how you handled errors and edge cases, such as empty lists or cases where no valid match is found.

## Task C: Filter Prime Numbers and Reverse Their Digits (40 Points)

Write a Prolog predicate filter\_and\_transform/3 that:

1. **Filters** elements from a list by checking if each number is a **prime number**.
2. **Transforms** the filtered prime numbers by **reversing the digits** of each number.
3. Ensures **tail-recursive optimization** using an accumulator.
4. Uses the **cut operator (!)** to stop the search after exactly **5 matching prime numbers** have been found and transformed.

Test Cases:

```
?- filter_and_transform([12, 18, 19, 20, 23, 25, 29, 31], Result).
```

```
Result = [91, 32, 92, 13] .
```

```
?- filter_and_transform([13, 24, 17, 29, 31, 37, 40, 41, 43], Result).
```

```
Result = [31, 71, 92, 13, 73] .
```

?- filter\_and\_transform([101, 103, 107, 109, 113, 127, 131, 137, 139], Result).  
Result = [101, 301, 701, 901, 311] .

?- filter\_and\_transform([4, 8, 10, 20, 25, 35], Result).  
Result = [].

?- filter\_and\_transform([], Result).  
Result = [].

?- filter\_and\_transform([103, 199, 373, 401, 997, 1009, 2011], Result).  
Result = [301, 991, 373, 104, 799] .

?- filter\_and\_transform([11, 14, 17, 19, 20], Result).  
Result = [11, 71, 91] .

?- filter\_and\_transform([23, 5, -7, 12, 17], Result).  
Result = [32, 5, 71] .

?- filter\_and\_transform([29], Result).  
Result = [92] .

?- filter\_and\_transform([2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31], Result).  
Result = [2, 3, 5, 7, 11] .

## Report Requirements:

In addition to submitting your code, please prepare a brief report covering the following aspects:

1. Describe the logic behind the prime-checking predicate and how it ensures that only prime numbers are selected.
2. Discuss the use of any helper predicates that handle the digit-reversing process.
3. Explain how your code uses Prolog's backtracking mechanism to stop after exactly 5 prime numbers are found and transformed.

## Lab Submission Guidelines

The due date for each lab is two weeks from the date of the lab session. For Lab 6, the submission deadline is **November 14<sup>th</sup>, 11.59 PM**. Each group is required to make a single submission on Brightspace.

1. One code file for each task in prolog file (Eg: filename.pl).
2. One PDF report explains your work concisely and includes a few screenshots of your output against the test case provided.
3. Zip everything into one folder and please upload it to Brightspace.

On the front page of your report, please include:

- Group number
- Names of all group members
- Student IDs

You will be evaluated as a group unless one of the members raises concerns about unequal contributions. In such cases, we may ask for further clarification and reserve the right to evaluate members individually.

If you use or receive help from ChatGPT or similar tools, you must clearly reference it in your submission. Failure to do so will result in a **zero** for the lab, and further actions may be taken in accordance with university policies.

You are allowed to use tools like ChatGPT for assistance, but **copying and pasting** the generated results directly into your assignment will not be tolerated. You are expected to **understand the help provided and implement the solution in your own words and approach**.